

Experiment 'p_5_4_middle' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_5_4_middle

Natural language statement: Write a method that returns the middle character of a string if the length is odd, or the two middle characters if the length is even. For example, middle("middle") returns "dd".

Method signature: p_5_4_middle(str: string) returns (middle_str: string)

Ensures

- if |str| % 2 == 1 then |middle_str| == 1 else |middle_str| == 2
- if |str| % 2 == 1 then middle_str[0] == str[|str| / 2] else middle_str == str[|str| / 2 - 1 .. (|str| / 2) + 1]

Requires

- 0 < |str|

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns the middle character of a string if the length is odd, or the
↪ two middle characters if the length is even. For example, `middle("middle")`
↪ returns `"dd"`.

The signature should be:

`p_5_4_middle(str: string)` returns `(middle_str: string)`

The method should respect the following contract:

requires $0 < |str|$, ensures if $|str| \% 2 == 1$ then $|middle_str| == 1$ else $|middle_str| ==$
↪ 2 , ensures if $|str| \% 2 == 1$ then $middle_str[0] == str[|str| / 2]$ else $middle_str$
↪ $== str[|str| / 2 - 1 .. (|str| / 2) + 1]$

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnSBzakaqzyL7SvcojBSjJmnHObh8

Model: gpt-5.5-2026-04-23

Created at: 1780678815

Finish reason: stop

Usage: 470 tokens in, and 1224 tokens out

```
{
  var n := |str|;
  var mid := n / 2;

  if n % 2 == 1 {
    assert n == 2 * mid + 1;
    assert mid < n;
    assert mid + 1 <= n;
    middle_str := str[mid .. mid + 1];
    assert |middle_str| == 1;
    assert middle_str[0] == str[mid];
  } else {
    assert n % 2 == 0;
    assert n == 2 * mid;
    assert mid > 0;
    assert mid + 1 <= n;
    middle_str := str[mid - 1 .. mid + 1];
  }
```

```
    assert |middle_str| == 2;
  }
}
```

Final Program

```
method p_5_4_middle(str: string) returns (middle_str: string)
  requires 0 < |str|
  ensures if |str| % 2 == 1 then |middle_str| == 1 else |middle_str| == 2
  ensures if |str| % 2 == 1 then middle_str[0] == str[|str| / 2] else middle_str ==
    ↪ str[|str| / 2 - 1 .. (|str| / 2) + 1]
{
  var n := |str|;
  var mid := n / 2;

  if n % 2 == 1 {
    assert n == 2 * mid + 1;
    assert mid < n;
    assert mid + 1 <= n;
    middle_str := str[mid .. mid + 1];
    assert |middle_str| == 1;
    assert middle_str[0] == str[mid];
  } else {
    assert n % 2 == 0;
    assert n == 2 * mid;
    assert mid > 0;
    assert mid + 1 <= n;
    middle_str := str[mid - 1 .. mid + 1];
    assert |middle_str| == 2;
  }
}
```

Total Token Usage

Input tokens: 470

Output tokens: 1224

Reasoning tokens: 1024

Sum of 'total tokens': 1694

Experiment Timings

Overall Experiment started at 1780678795916, ended at 1780678910818, lasting 114902ms (114.90 seconds)

Iteration #1 started at 1780678795919, ended at 1780678910814, lasting 114895ms (114.90 seconds)